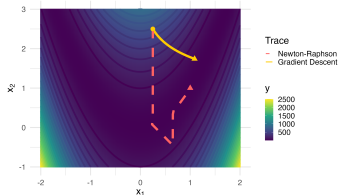
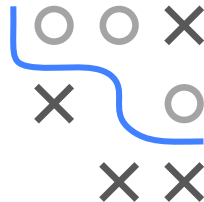


Optimization in Machine Learning

Second order methods

Newton-Raphson



Learning goals

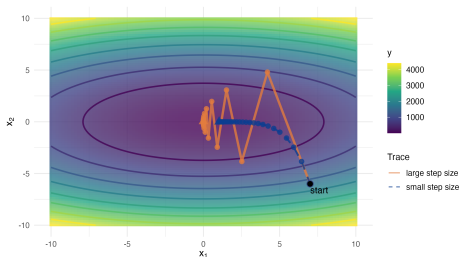
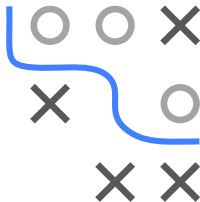
- Newton-Raphson
- Limitations

Slides based on

► Aggarwal 2020 (Ch. 5.4-5.7)

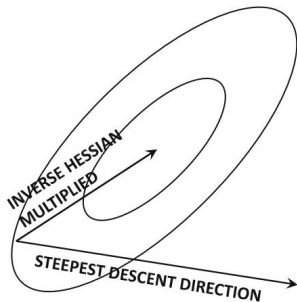
MOTIVATION: BEYOND GRADIENT INFORMATION

- **Recall:** GD only uses the gradient $\Rightarrow$ *ignores curvature*
- Hessian $\mathbf{H} = \nabla^2 f$ (curvature) encodes how fast the gradient itself changes along each direction
 - High curvature: gradient is unreliable further away $\Rightarrow$ take a *small* step
 - Low curvature: gradient stays valid further away $\Rightarrow$ take a *large* step
- GD only picks one global α for all directions $\Rightarrow$ for ill-conditioned $\mathbf{H}$, this causes zig-zagging / slow convergence:



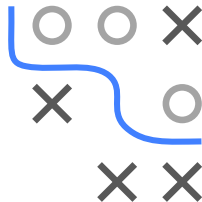
MOTIVATION: BEYOND GRADIENT INFORMATION

- **Idea:** Use full curvature information instead of a single scalar α
 - rescale the gradient *individually per direction* via $\mathbf{H}^{-1}$
 - biases updates towards low-curvature directions, where a large step is unlikely to overshoot or worsen f



► [Click for source](#)

Effect of multiplying the steepest-descent direction with the inverse Hessian



NEWTON-RAPHSON

- **Assumption:** $f \in \mathcal{C}^2$
- **Aim:** Find stationary point $\mathbf{x}^*$, i.e., $\nabla f(\mathbf{x}^*) = \mathbf{0}$
- **Idea:** Find root of first order Taylor approx of $\nabla f(\mathbf{x})$:

$$\nabla f(\mathbf{x}) \approx \nabla f(\mathbf{x}^{[t]}) + \nabla^2 f(\mathbf{x}^{[t]})(\mathbf{x} - \mathbf{x}^{[t]}) = \mathbf{0}$$

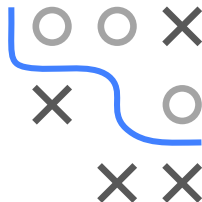
$$\nabla^2 f(\mathbf{x}^{[t]})(\mathbf{x} - \mathbf{x}^{[t]}) = -\nabla f(\mathbf{x}^{[t]})$$

$$\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} - (\nabla^2 f(\mathbf{x}^{[t]}))^{-1} \nabla f(\mathbf{x}^{[t]})$$

- **Update scheme:**

$$\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} + \mathbf{d}^{[t]}$$

$$\text{with } \mathbf{d}^{[t]} = -(\nabla^2 f(\mathbf{x}^{[t]}))^{-1} \nabla f(\mathbf{x}^{[t]})$$



NEWTON-RAPHSON: PRACTICE

- **Note:** In practice, we get $\mathbf{d}^{[t]}$ by solving the linear system

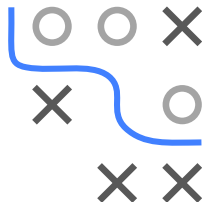
$$\nabla^2 f(\mathbf{x}^{[t]})\mathbf{d}^{[t]} = -\nabla f(\mathbf{x}^{[t]})$$

with direct (matrix decompositions) or iterative methods

- **Relaxed/Damped Newton-Raphson:** Use step size $\alpha > 0$ with

$$\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} + \alpha \mathbf{d}^{[t]}$$

to satisfy Wolfe conditions (or just Armijo rule)



ANALYTICAL EXAMPLE WITH QUADRATIC FORM

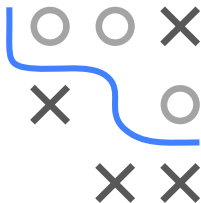
- Consider $f(x_1, x_2) = x_1^2 + \frac{x_2^2}{2}$
- Update direction: $\mathbf{d}^{[t]} = -(\nabla^2 f(x_1^{[t]}, x_2^{[t]}))^{-1} \nabla f(x_1^{[t]}, x_2^{[t]})$
- Gradient and Hessian:

$$\nabla f(x_1, x_2) = \begin{pmatrix} 2x_1 \\ x_2 \end{pmatrix}, \quad \nabla^2 f(x_1, x_2) = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$$

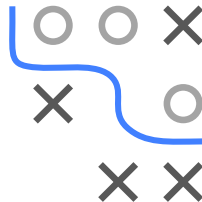
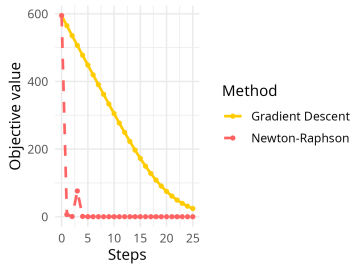
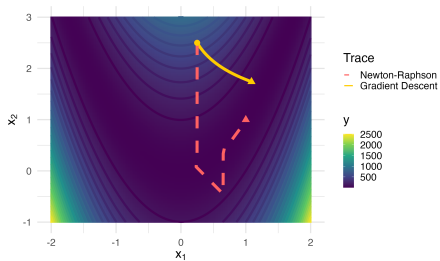
- First step:

$$\begin{aligned} \begin{pmatrix} x_1^{[1]} \\ x_2^{[1]} \end{pmatrix} &= \begin{pmatrix} x_1^{[0]} \\ x_2^{[0]} \end{pmatrix} + \mathbf{d}^{[0]} = \begin{pmatrix} x_1^{[0]} \\ x_2^{[0]} \end{pmatrix} - \begin{pmatrix} 1/2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2x_1^{[0]} \\ x_2^{[0]} \end{pmatrix} \\ &= \begin{pmatrix} x_1^{[0]} \\ x_2^{[0]} \end{pmatrix} + \begin{pmatrix} -x_1^{[0]} \\ -x_2^{[0]} \end{pmatrix} = \mathbf{0} \end{aligned}$$

- **Note:** Newton-Raphson only needs one iteration for quadratic forms



NEWTON-RAPHSON VS. GD ON ROSENBROCK FUNCTION



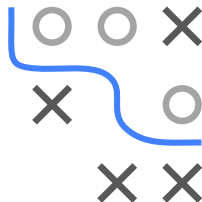
Newton-Raphson has much better convergence speed here

LIMITATIONS: NON-DESCENT DIRECTIONS

- **Problem 1:** In general, $\mathbf{d}^{[t]}$ is not a descent direction
- **But:** If Hessian is positive definite, $\mathbf{d}^{[t]}$ is descent direction:

$$\nabla f(\mathbf{x}^{[t]})^\top \mathbf{d}^{[t]} = -\nabla f(\mathbf{x}^{[t]})^\top (\nabla^2 f(\mathbf{x}^{[t]}))^{-1} \nabla f(\mathbf{x}^{[t]}) < 0$$

- Near minimum, Hessian is positive definite
- For initial steps, Hessian is often not positive definite and Newton-Raphson may give non-descending update directions

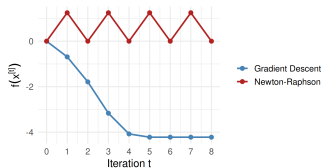
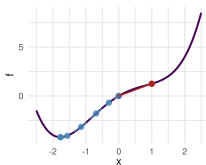
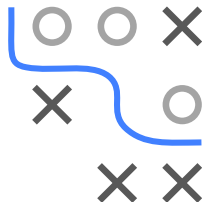


EXAMPLE: NON-DESCENT DIRECTIONS

- Consider $f(x) = \frac{x^4}{4} - x^2 + 2x$ with $f'(x) = x^3 - 2x + 2$, $f''(x) = 3x^2 - 2$
- Starting at $x^{[0]} = 0$: $f''(0) = -2 < 0$ (**H** not pd) $\Rightarrow$ Newton-Raphson performs **ascent** step:

$$x^{[1]} = x^{[0]} - \frac{f'(x^{[0]})}{f''(x^{[0]})} = 0 - \frac{2}{-2} = 1, \quad f(x^{[1]}) = 1.25 > 0 = f(x^{[0]})$$

- From $x^{[1]} = 1$: $f''(1) = 1 > 0$, but the step leads right back to $x^{[2]} = 0$
 $\Rightarrow$ Newton-Raphson cycles forever between 0 and 1
- GD** from the same $x^{[0]} = 0$ only uses the neg. gradient $-f'(x^{[0]}) = -2$ (descent direction) and converges for appropriate step size α



LIMITATIONS: COMPUTATIONAL COST

- **Problem 2:** Hessian can be **computationally expensive** to calculate, as descent direction $\mathbf{d}^{[t]}$ is solution of the linear system

$$\nabla^2 f(\mathbf{x}^{[t]})\mathbf{d}^{[t]} = -\nabla f(\mathbf{x}^{[t]})$$

- For $\mathbf{x} \in \mathbb{R}^d$, the Hessian has d^2 entries: $\mathcal{O}(d^2)$ memory, and $\mathcal{O}(d^3)$ per iteration to solve the linear system (compared to $\mathcal{O}(d)$ memory and no linear solve for GD)
- For large d (e.g., deep neural networks with $d \sim 10^9$ parameters), this is prohibitive: storing a dense $d \times d$ Hessian alone is infeasible
- **Aim:** Find quasi-second order methods not relying on exact Hessians
 - Quasi-Newton method
 - Gauss-Newton algorithm (for least squares)

